PROJECT BROCHURE

Health Care Assistant

A clinical AI support platform for structured analysis, controlled access, conversation traceability, and operational governance.

Why it matters

Health Care Assistant brings secure user onboarding, multi-turn AI conversations, clinical-context intake, model governance, and credit-based usage control into a single Next.js application. It is designed for organizations that want practical AI support workflows without losing visibility over who can access the system, which models are used, and how activity is recorded.

Next.js 14 App Router | Supabase + PostgreSQL | Anthropic Claude | Cloudflare R2 | JWT + OTP + Google OAuth

3

ACCESS PATHS

Password, email OTP, and Google OAuth help lower adoption friction while keeping controlled onboarding.

4

CLINICAL MODES

Lab, radiology, medical records, and medication analysis align the workspace to common healthcare use cases.

1

GOVERNED PLATFORM

Identity, AI interaction, storage, pricing, and admin control are managed within one application boundary.

Built for clinical support workflows. It should operate under organizational privacy, security, and professional-review policies rather than as a standalone diagnosis system.

EXECUTIVE SUMMARY

Key Capabilities

The platform combines end-user workflow features with the controls healthcare teams typically need for adoption, oversight, and staged rollout.

Flexible Access and Approval

Users can sign in with password, email OTP, or Google OAuth. New accounts stay under approval control before clinical chat functions become available.

Clinical AI Workspace

The main workspace supports multi-turn conversations, selectable workload depth, and clinical analysis modes for lab, radiology, records, and medication scenarios.

File and Context Intake

The solution accepts common document and image formats, supports screenshots, and includes FHIR-oriented import and formatting workflows for richer context.

Conversation Traceability

Persistent conversation history, downloadable logs, and saved message records help teams revisit prior AI discussions and keep an auditable trail.

Model and Cost Governance

Credits are checked before AI use, deducted per model policy, and written to a transaction ledger. Admins can manage model availability and pricing.

Organization-ready Experience

Customer-specific UI settings, bilingual interface support, and role-based admin functions make the platform easier to align with institutional workflows.

OPERATING MODEL

Architecture at a Glance

The application follows a full-stack Next.js design: the browser experience, API routes, business logic, and integrations are organized within one codebase for simpler delivery and consistent governance.

Simple Delivery Flow

This simplified flow illustrates how a governed clinical AI request moves through the platform.

Step 1

Authenticate

Verify identity, session, and approval status before protected workflows begin.

→

Step 2

Prepare Context

Collect prompt, workload level, clinical mode, model selection, and optional files.

→

Step 3

Run AI Policy

Validate input, enforce credit rules, and route the request through the AI client layer.

→

Step 4

Record Results

Store messages, update credits, and retain conversation history for follow-up and review.

Client Experience

- Login and registration pages
- Chat workspace and conversation history
- Admin dashboard and settings-driven UI
- Traditional Chinese and

Application Core

- Next.js App Router pages and route handlers
- Validation with Zod and session checks with JWT cookies
- Workload mapping and clinical function routing
- Conversation, credit, and model

Data and Services

- Supabase for users, sessions, messages, credits, and pricing
- Anthropic Claude for AI responses
- Resend for OTP email and Google OAuth for federated sign-in

English interface support

orchestration logic

- Cloudflare R2 for uploaded files and artifacts

User request → Controlled AI processing → Recorded outcome

01
Authenticate the user and verify account status.

02
Select workload depth, analysis mode, model, and optional files.

03
Validate payloads and resolve credit and pricing policy.

04
Send structured context to Claude through the AI client layer.

05
Save messages, update credits, and keep history available.

Technology Stack

React 18 | TypeScript 5 | Tailwind CSS | Supabase | PostgreSQL | jose (JWT) | bcryptjs | Zod | Resend | Google OAuth | Cloudflare R2 | Vitest

---

Built for clinical support workflows. It should operate under organizational privacy, security, and professional-review policies rather than as a standalone diagnosis system.

GOVERNANCE

Security, Governance, and Operations

Beyond the chat interface, the project emphasizes controlled access, predictable cost management, and operational visibility.

Session and Access Control

JWT sessions are stored in HttpOnly cookies. Approval status and admin role checks help enforce who can enter protected workflows.

Validation and Abuse Protection

API payloads are validated with Zod, passwords are hashed with bcrypt, and rate limits reduce OTP and login abuse risk.

Operational Transparency

Credits, pricing, customer records, and model states are surfaced through admin APIs so teams can monitor adoption and usage behavior.

Ideal Use Cases

- Clinical support conversations around lab reports and structured observations
- Radiology-oriented discussions with image or document attachments
- Medical record summarization with controlled conversation history
- Medication review workflows with model-cost awareness

Deployment Value

Health Care Assistant is suitable for teams that want one governed platform covering authentication, AI interactions, storage, and cost tracking. The current architecture is well positioned for phased rollout, customer-level customization, and future healthcare workflow extensions.